Moodle Data API — v1.3.1

Read-only JSON API exposing users, courses, categories, enrolments, activities, grades, quizzes, cohorts and dashboard stats from a Moodle database. **Login is restricted to Moodle administrators.**

- **Base URL:** `/api`
- **Interactive docs (Swagger UI):** `/docs/`
- **OpenAPI spec:** `/docs/openapi.json`

Table of contents

Authentication

Every endpoint except `POST /api/auth/login` requires a bearer token:

```
Authorization: Bearer <token>
```

Tokens are issued by `POST /api/auth/login` and are valid for **8 hours**. Use `POST /api/auth/logout` to revoke the current token.

Admin gate

A user is considered an admin if **either** of these is true:

1. They are a Moodle site administrator (top-level admin set via Moodle's UI, stored as a comma-separated user-id list in the config table):

```
SELECT value FROM mdl_config WHERE name = 'siteadmins';
-- e.g. "37,8,9,1592,7,1814,2676,..."  → user.id matches any of these
```

2. They have the `manager` or `administrator` role at the system context (`contextlevel = 10`):

```
SELECT 1
FROM mdl_user u
JOIN mdl_role_assignments ra ON ra.userid = u.id
JOIN mdl_role r              ON r.id = ra.roleid
JOIN mdl_context ctx         ON ctx.id = ra.contextid
WHERE r.shortname IN ('manager', 'administrator')
  AND ctx.contextlevel = 10
  AND u.id = :user_id
```

The site-admin check is evaluated first; the role check runs only if it fails. Both checks together cover every privileged user in Moodle (site admins are not duplicated in `role_assignments`).

If the user authenticates successfully but neither check passes, the API responds:

```
HTTP 422
{
  "message": "This account is not authorized for admin access.",
  "errors": { "username": ["This account is not authorized for admin access."] }
}
```

Empty passwords (typically SSO-only accounts) are also rejected.

Login is rate-limited

`POST /api/auth/login` is throttled to **5 requests per minute per IP**. Excess requests return `429 Too Many Requests`.

Quick start

```
# 1. Log in
curl -X POST http://localhost:8000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin-user","password":"secret"}'
# → { "token": "1|abcdef...", "expires_at": "...", "user": { ... } }

# 2. Use the token
curl http://localhost:8000/api/users \
  -H 'Authorization: Bearer 1|abcdef...'
```

Conventions

Pagination

Every list endpoint supports:

Query param	Default	Notes
`page`	`1`	Page number
`per_page`	`15` – `50` (varies)	Max `200`

Paginated responses use this envelope:

```
{
  "data": [ /* items */ ],
  "links": {
    "first": "...?page=1",
    "last":  "...?page=N",
    "prev":  null,
    "next":  "...?page=2"
  },
```

```
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 144,
    "path": "http://localhost/api/users",
    "per_page": 15,
    "to": 15,
    "total": 2152
  }
}
```

Single-item responses use `{ "data": { ... } }`.

Timestamps

All Moodle unix timestamps are converted to ISO 8601 (`2026-04-16T14:43:34+00:00`).

Hidden fields

`User`: `password`, `secret`, `lastip` are never returned. `Enrol`: `password` is never returned.

Error format

```
{
  "message": "Resource not found"
}
```

`422` validation errors include an `errors` object keyed by field name.

Endpoints

Every endpoint requires `Authorization: Bearer <token>` unless noted otherwise. Examples assume the token has been obtained via `POST /api/auth/login`.

Auth

`POST /api/auth/login` — *no auth required*

Issue a token for an admin user.

Body

```
{
  "username": "string (required)",
  "password": "string (required)",
  "device_name": "string (optional, default: \"api\")"
}
```

Response 200

```
{
  "token": "1|JZz9oWxMYFUiYaQy5oWEzMDGyLdeQq3RVwoDNMEoba68...",
  "expires_at": "2026-04-16T22:43:40+00:00",
  "user": { /* User */ }
}
```

Errors: `422` (bad credentials or non-admin), `429` (rate-limited).

`POST /api/auth/logout`

Revoke the current bearer token.

Response 200

```
{ "message": "Logged out." }
```

`GET /api/auth/me`

Return the authenticated admin.

Response 200 — `{ "data": User }`

Users

`GET /api/users`

Paginated, non-deleted users.

Query

Param	Type	Notes
`search`	string	Matches `username`, `email`, `firstname`, `lastname`
`sort`	enum	`id` (default), `username`, `email`, `firstname`, `lastname`, `lastaccess`, `timecreated`
`direction`	enum	`asc` (default), `desc`
`page`, `per_page`	int	Pagination

Response 200 — paginated list of `User`.

`GET /api/users/{id}`

Single user. `404` if missing or deleted.

`GET /api/users/{id}/courses`

Paginated list of `Course` records the user is enrolled in.

`GET /api/users/{id}/grades`

Paginated list of `Grade`. Optional `?course_id=` filter.

`GET /api/users/{id}/quiz-attempts`

Paginated list of `QuizAttempt`. Optional `?state=inprogress|overdue|finished|abandoned`.

Courses

`GET /api/courses`

Paginated courses (excludes Moodle site course id=1).

Query

Param	Type	Default	Notes
`search`	string	—	Matches `fullname`, `shortname`, `idnumber`
`category_id`	int	—	Filter by category
`visible_only`	bool	`true`	Hide invisible courses
`page`, `per_page`	int	—	Pagination

`GET /api/courses/{id}`

Single course (includes nested `category`).

`GET /api/courses/{id}/students`

Paginated `User` enrolled in the course.

`GET /api/courses/{id}/activities`

Paginated `Activity` (course modules: quizzes, assignments, labels, etc.).

`GET /api/courses/{id}/grades`

Paginated `Grade` for the course.

Categories

`GET /api/categories`

Paginated category list.

Query

Param	Type	Default	Notes
`tree`	bool	`false`	If `true`, returns root categories with nested `children` (3 levels deep)

Param	Type	Default	Notes
`page`, `per_page`	int	—	Pagination

`GET /api/categories/{id}`

Single category (includes immediate `children`).

Quizzes

`GET /api/quizzes`

Paginated quizzes.

Query

Param	Type	Notes
`search`	string	Matches quiz `name`
`course_id`	int	Filter by course
`page`, `per_page`	int	Pagination

`GET /api/quizzes/{id}`

Single quiz.

`GET /api/quizzes/{id}/attempts`

Paginated `QuizAttempt` for the quiz. Optional `?state=`, `?user_id=`.

Cohorts

`GET /api/cohorts`

Paginated cohorts.

Query

Param	Type	Default
`search`	string	—
`visible_only`	bool	`true`
`page`, `per_page`	int	—

`GET /api/cohorts/{id}`

Single cohort.

`GET /api/cohorts/{id}/members`

Paginated `User` belonging to the cohort.

Stats

All stats endpoints are **cached for 10 minutes**.

`GET /api/stats/overview`

Platform-wide totals.

Response 200


```
{
  "data": {
    "users":       { "total": 2152, "suspended": 0, "active_last_30d": 136, "active_last_7d":
    "courses":     { "total": 42, "visible": 14, "categories": 10 },
    "enrolments":  { "total": 1827, "active": 1827 },
    "completions": { "started": 1804, "completed": 388 },
    "quizzes":     { "total": 316, "attempts_total": 8010, "attempts_finished": 7223 },
    "cohorts": 8,
    "generated_at": "2026-04-16T14:43:34+00:00"
  }
}
```


`GET /api/stats/courses/{id}`

Per-course metrics: students, activities, quizzes, attempts, average final grade, completion rate.


```
{
  "data": {
    "course_id": 10,
    "students": 96,
    "activities": 68,
    "quizzes": 19,
    "quiz_attempts": 378,
    "average_grade": 35.46,
    "grade_max": 130,
    "graded_users": 39,
    "completions": { "started": 97, "completed": 7, "rate": 0.0722 },
    "generated_at": "..."
  }
}
```


`GET /api/stats/users/{id}/activity`

Per-user activity: last access, enrolment counts, average grade, recent log events.

Query

Param	Type	Default	Max
`events`	int	`20`	`100`


```
{
  "data": {
    "user_id": 10,
    "last_access_at": "2026-03-07T16:28:34+00:00",
    "last_login_at":  "2026-02-27T15:37:20+00:00",
    "enrolled_courses": 2,
    "completed_courses": 0,
    "average_grade": 2.6,
    "recent_events": [
      {
        "id": 1720216,
```

```
        "event": "\\core_h5p\\event\\h5p_viewed",
        "action": "viewed",
        "target": "h5p",
        "course_id": 54,
        "origin": "web",
        "occurred_at": "2026-03-07T16:29:17+00:00"
      }
    ],
    "generated_at": "..."
  }
}
```

Schema reference

`User`

`id`, `username`, `email`, `first_name`, `last_name`, `full_name`, `id_number`, `phone1`, `phone2`, `institution`, `department`, `city`, `country`, `lang`, `timezone`, `suspended` (bool), `deleted` (bool), `confirmed` (bool), `first_access_at`, `last_access_at`, `last_login_at`, `created_at`, `updated_at`.

`Course`

`id`, `full_name`, `short_name`, `id_number`, `category_id`, `category` (nested `Category`), `summary`, `format`, `visible` (bool), `lang`, `start_date`, `end_date`, `created_at`, `updated_at`.

`Category`

`id`, `name`, `id_number`, `parent_id`, `depth`, `path`, `course_count`, `visible` (bool), `children` (array of `Category`).

`Grade`

`id`, `item_id`, `user_id`, `item_name`, `item_type`, `course_id`, `raw_grade`, `final_grade`, `grade_max`, `grade_min`, `grade_pass`, `feedback`, `updated_at`.

`Activity`

`id`, `section`, `type` (e.g. `quiz`, `assign`, `label`), `instance_id`, `visible` (bool).

`Quiz`

`id`, `name`, `course_id`, `intro`, `time_open`, `time_close`, `time_limit_seconds`, `attempts_allowed` (0 = unlimited), `grade_method`, `grade_max`, `sum_grades`, `created_at`, `updated_at`.

`QuizAttempt`

`id`, `quiz_id`, `user_id`, `attempt_number`, `state` (`inprogress` / `overdue` / `finished` / `abandoned`), `preview` (bool), `sum_grades`, `started_at`, `finished_at`, `updated_at`.

`Cohort`

`id`, `name`, `id_number`, `description`, `visible` (bool), `context_id`, `created_at`, `updated_at`.

Operational notes

- The Moodle MySQL connection is **read-only**. Sanctum tokens, sessions, and cache are stored in a separate SQLite database (`database/auth.sqlite`).
- The user table is named `user` (singular) in this Moodle install; `DB_PREFIX=mdl_` is applied automatically by Laravel.
- All stats endpoints aggregate from `mdl_logstore_standard_log` (1.7M rows) and are cached to keep response times fast.